Python Debugging — Worked Example

> Back to the exercise · Round 1: Basic · Round 2: Intermediate · Round 3: Real-world · Round 4: Multi-agent · Solution

One scenario, followed across all 4 rounds: Project 1's chatbot history-handling code. Each round makes the same underlying kind of mistake (a plain Python gotcha) show up in a harder place to spot. Work through them in order — don't jump ahead. Each round ends with **"What do you think is wrong?"** — actually stop and answer before reading on.

- Round 1: Basic
- Round 2: Intermediate
- Round 3: Real-world
- Round 4: Multi-agent

Round: Basic

Setup: Project 1's chatbot keeps its whole conversation in a list. Before every call to the model, a helper trims it down to the last `max_turns` messages:

```
def trim_history(history, max_turns):
    trimmed = history[-max_turns:]
    if len(trimmed) % 2 != 0:
        trimmed = trimmed[1:]
```

Symptoms: the chatbot crashes on the very first message you send it, every single time — not intermittent.

Error output:

```
Traceback (most recent call last):
  File "project_1_supportdesk_chat_and_triage/chatbot.py", line 42,
      in send_message
    for msg in trim_history(conversation, MAX_TURNS):
TypeError: 'NoneType' object is not iterable
```

Expected vs. actual:

- Expected: `trim_history()` returns a list of the most recent messages, ready to loop over.

- Actual: it crashes immediately — `trim_history(...)` appears to be `None`, every time it's called.

What do you think is wrong?

Round: Intermediate

Setup: `trim_history()` got fixed. Now a different helper attaches a short debug note to each turn, so you can see what a session looked like later:

```
def log_turn(role, text, notes=[]):
    notes.append(role + ": " + text[:40])
    return notes
```

It's called once per turn: `session_notes = log_turn("user", user_text)`.

Symptoms: this looks like a *session isolation* bug, not a Python bug — two completely different users' chatbot sessions, running one after another in the same long-running process, end up sharing debug notes.

Error / log output:

```
[session abc123] notes so far:
  ['user: what is the refund policy?', 'user: how do I reset my password?']
```

Session `abc123` only ever asked about resetting a password. The refund-policy line came from session `xyz789`, which ran earlier in the same process and finished minutes ago.

Expected vs. actual:

- Expected: each session's notes list starts empty and only ever contains that session's own turns.
- Actual: notes from unrelated, already-finished sessions keep showing up in later sessions' notes — but only when the app has been running a while, not on a fresh restart.

What do you think is wrong?

Round: Real-world

Setup: a shared in-memory response cache (used by Doc02's `http_client.py` callers) has a cleanup function that drops expired entries:

```
def clear_expired(cache, now):
    for key in cache:
```

```
        if cache[key]["expires_at"] < now:
            del cache[key]
```

Symptoms: it only sometimes happens. It passed every manual test you ran while building it. In production, it crashes maybe once every hour or two.

Error / log output:

```
Traceback (most recent call last):
  File "cache_utils.py", line 15, in clear_expired
    for key in cache:
RuntimeError: dictionary changed size during iteration
```

Expected vs. actual:

- Expected: whatever expired entries exist get removed safely, whether there are zero of them or several.
- Actual: it works fine every time you tested it right after populating a fresh cache — because nothing had expired yet. It only fails once the cache genuinely has at least one expired entry at cleanup time, which in production happens constantly but almost never happened in your quick manual tests.

What do you think is wrong?

Round: Multi-agent

Python bugs don't stop being Python bugs just because they're now inside a LangGraph node — they just get harder to see, because a unit test that calls a node function once, in isolation, often can't trigger them at all. This round is the Round 2 bug again, in Project 4.

Setup: Project 4's supervisor node collects a short list of routing notes, one call per incoming request, using the same pattern as Round 2:

```
def route(state, notes=[]):
    notes.append(state["task"])
    next_agent = pick_agent(state)
    return Command(goto=next_agent, update={"routing_notes": notes})
```

`test_route()` — a unit test that builds one fake `state`, calls `route(state)` once, and checks the `Command` it gets back — passes every time.

Symptoms: running the full 4-agent pipeline against several real requests back to back (not a single test call), the Reviewer starts rejecting drafts as "confusing" — its rejection reason quotes task

descriptions that have nothing to do with the current request.

Log output:

```
[request #5] routing_notes: [
  'summarize Q1 sales report',
  'draft a product announcement',
  'answer a support ticket about billing',
  'check compliance wording on a contract',
  'write a release blog post'
]
[reviewer] REJECTED: routing_notes references unrelated tasks,
  draft may be confused
```

Request #5 was only ever "write a release blog post." The other four lines are requests #1 through #4, run earlier in the same process.

Expected vs. actual:

- Expected: `routing_notes` for a given request contains only that request's own routing history.
- Actual: it silently accumulates every request's task description, for the life of the running process — invisible in `test_route()`, because that test only ever calls `route()` once before the process (and the test) ends.

What do you think is wrong?

Full solution: Show me the solution